



REDLINE DAYLIGHT • AUTOMATION HANDBOOK

الدليل الشامل لتعلم Ansible

مرجعك الكامل لـ **Ansible** — الـ Architecture، التثبيت على كل الأنظمة، إعداد الـ SSH Keys، الـ Modules والـ Inventory، الـ Ad-hoc Commands، أساسيات الـ YAML، الـ Playbooks بالكامل مع قراءة النتائج، والـ Roles مع منصة Galaxy — بناءً على شرح قناة **Cloud Code**، تفريغًا وتصحيحًا وتنظيمًا كمرجع مكتوب متكامل. كل المصطلحات التقنية بالإنجليزية كما هي في بيئة العمل.

By Nabawy

AUTHOR & COMPILER • إعداد وتنسيق

Roles

تنظيم احتراقي

YAML

لغة الـ Playbooks

SSH

اتصال آمن

REDLINE EDITION

نسخة نهائية • A4-Fixed

المحتويات

11	Inventory — ملف ال Servers	08	3	الخريطة — ملخص تنفيذي	01
12	Ad-hoc Commands — التنفيذ الفوري	09	4	Intro — مقدمة ومميزات Ansible	02
13	YAML Basics — أساسيات الصيغة	10	5	Architecture — المعمارية	03
14	Playbooks — كتابة وتشغيل	11	6	Comparison — مقارنة طرق الإدارة	04
15	Playbooks — الإزالة وقراءة النتائج	b•11	7	Use Cases — مجالات الاستخدام	05
16	Roles & Galaxy — التنظيم	12	8	Installation — التثبيت وإعداد البيئة	06
17	اختبر نفسك — 12 سؤالًا	🎯	9	SSH Keys — الاتصال بدون Password	07
18	مفتاح الإجابات	✓	10	Modules — وحدات التنفيذ	
19	الورقة المرجعية	✳️			

ok Server بالحالة المطلوبة، لا تغيير changed تم تنفيذ تغيير بنجاح failed حدث خطأ في ال Task RED عناوين وهيكل (ديكور بنيوي)	status legend TASK [دليل ألوان الحالة]
--	---

[DEBUG] كيف تستخدم الكتاب: 12 فصلًا مترابطًا — كل فصل يفتح بما قبله ويجهّزك لما بعده. كل المصطلحات التقنية بالإنجليزية كما في بيئة العمل (Agentless, Playbook, Inventory...) والشرح بالعربية. الأكواد تُقرأ من اليسار والنص من اليمين. أي سطر مفقود من ملفك الأصلي موجود هنا — قارن بحرية.

الملخص التنفيذي — الطريق كاملاً

قبل أن نلمس أي سطر أوامر، هذه هي القصة كاملة — كل جملة رابط يأخذك لفصلها مباشرة.

نبدأ بسؤال وجودي: لماذا تُتعب أنفسنا بالدخول لكل **Server** يدويًا؟ الإجابة تبدأ في **الفصل 1 — Intro: Ansible** أداة **Automation** مفتوحة المصدر (**Open-Source**)، بدون عملاء **Agentless**، بصيغة **YAML** بسيطة، بأسلوب **Declarative**، وأمنة على التكرار **Idempotent**. ثم نرى في **الفصل 2 — Architecture** الصورة بُعديها: **Control Node** يهمس عبر **SSH Connection** بزواج **Public/Private Keys** إلى **Managed Nodes** (Ubuntu / CentOS / DB).

ثم في **الفصل 3 — Comparison** نضع Ansible في الميزان ضد الطريقة **Manual** وسكريبتات **Bash** على 3 Servers مختلفة التوزيعات (Ubuntu، CentOS، Alpine) لتثبيت **Nginx**. وفي **الفصل 4 — Use Cases** نستعرض المجالات الثلاثة: **Configuration**، **Network Automation**، و**Management, Infrastructure Provisioning** (ومن هنا نتحدث عن Terraform).

الآن العدة العملية: في **الفصل 5 — Installation** نثبت على macOS و Ubuntu و Pip و WSL، ونجهّز الـ VS Code Extensions. وفي **الفصل 6 — SSH Keys** ننشئ الـ Key Pair بـ `ssh-keygen` ونرفع الـ Public Key بـ `ssh-copy-id` — الأساس الذي تقف عليه كل أتمتة قادمة.

اللغة والتنفيذ: في **الفصل 7 — Modules** نتعرّف على `apt/yum` و `ping` و `service` و `copy` و `command/shell`. وفي **الفصل 8 — Inventory** نكتب ملف `hosts.ini` بالمجموعات. ثم في **الفصل 9 — Ad-hoc Commands** نفحص وننفذ فورًا. وفي **الفصل 10 — YAML** نتقن الـ Indentation وهياكل البيانات.

الذروة: في **الفصل 11 — Playbooks** نكتب **Playbook** تثبيت وتشغيل **Apache** كاملاً ونشغله ونقرأ الـ Recap، وفي **الفصل 11-b** نعكسه بالإزالة. وأخيرًا في **الفصل 12 — Roles & Galaxy** ننظم ونشارك. اختتم بـ **الاختبار والمرجع السريع**. الطريق واضح. ▶

KEY IDEA الفكرة الجامعة: أنت تكتب الـ **Desired State** مرة واحدة، و**Ansible** يحققها على كل الـ **Managed Nodes** — وإن لم تكن بحاجة فعلية للتغيير، فلن يلمس شيئًا.

Ansible هي أداة **Automation** مفتوحة المصدر (**Open-Source**) تُستخدم بشكل أساسي في مجالين:

2 Infrastructure Provisioning

بناء وتجهيز البنية التحتية — إنشاء الـ Servers والموارد السحابية.

1 Configuration Management

إدارة الإعدادات — تثبيت وتحديث البرامج وضبط الملفات على الـ Servers.

◀ **TASK** [أهم مميزات Ansible الأساسية] the four pillars

1.1 Agentless — بدون عميل

لا تحتاج لتثبيت أي برامج أو **Agents** على الـ **Managed Nodes** المستهدفة.

🔑 الاتصال يتم عبر بروتوكول **SSH** (في أنظمة Linux) أو **WinRM/SSH** (في أنظمة Windows).

🔄 المتطلب الوحيد على Servers الـ Linux هو وجود بيئة **Python**.

1.2 Simple Syntax — صيغة بسيطة

تُكتب جميع الإعدادات وسيناريوهات التشغيل بلغة **YAML** — وهي لغة قراءة وكتابة بسيطة جدًا وسهلة التعلم والتنفيذ. لا أقواس معقدة ولا فاصيل منقوطة؛ فقط مسافات وترتيب.

1.3 Declarative Approach — الأسلوب التوضيفي

تحدد في ملف الـ **YAML** الـ **Desired State** (الحالة المطلوبة) — مثل: «أريد تثبيت Apache وGit» — ويقوم Ansible بتنفيذ الـ Steps اللازمة للوصول لهذه الحالة بنفسه. ويدعم أيضًا الـ **Imperative Approach** عند استخدام الـ **Ad-hoc Commands**.

1.4 Idempotency — التكرار الآمن

تعني أن تنفيذ نفس الـ Script عدة مرات متتالية لن يغيّر شيئًا إلا إذا كانت هناك حاجة فعلية للتغيير.

[**EXAMPLE**] إذا طلبت من Ansible تثبيت Apache وكان البرنامج مثبتًا بالفعل بنفس الإصدار → لن يعيد تثبيته ولن يحدث أي تغيير على الـ Server. ستظهر الحالة **ok**. غير مثبت؟ يثبتته → **changed**.

Idempotent

Declarative

Simple YAML

Agentless

🔗 كيف تُبنى هذه الأداة داخليًا؟ مكوّنات فقط — **الفصل 2: Architecture**

تتكون معمارية Ansible من **مكوّنين رئيسيين** — بساطة متعمدة هي سر قوتها.

[Architecture - المكوّنان والقناة بينهما]



2.1 Control Node — عقدة التحكم

الجهاز أو ال Server الذي يتم تثبيت **Ansible** عليه، ومن خلاله يقوم مهندس ال **DevOps** بتشغيل الأوامر وكتيبات التشغيل (**Playbooks**). يمكن أن يكون Laptop شخصيًا أو Linux Server في معمل أو Cloud.

2.2 Managed Nodes (Targets) — العقد المُدارة

ال Servers المستهدفة بتطبيق التغييرات والإعدادات. تتلقى ال Instructions عبر **SSH** وتنفذها بدون الحاجة لتثبيت **Agents** عليها — لهذا نسمي المعمارية **Agentless**.

[**KEY IDEA**] لاحظ ما لا يوجد: لا Database مركزية، لا Daemon يعمل باستمرار على ال Targets، لا Agent يُحدّث. Ansible يتصل عند الحاجة، ينقذ، ثم يرحل. العبء على ال Managed Nodes = صفر.

TASK [خريطة المكونات] component map

المكوّن	ما هو؟	ماذا يُنثَب عليه؟	دوره
Control Node	Laptop / Linux Server	Ansible (وحده)	تشغيل الأوامر وال Playbooks
Managed Nodes	...Ubuntu / CentOS / DB	لا شيء — Python فقط	استقبال أوامر SSH وتنفيذها

[**WATCH OUT**] خطأ شائع عند المبتدئين: محاولة تثبيت Ansible على ال Managed Nodes «احتياطيًا». لا تفعل — هي لا تحتاجه إطلاقًا. ال Control Node هو المكان الوحيد للتثبيت (سنعود له في فصل ال Installation).

EXAM سؤال المقابلة: «صف معمارية Ansible؟» → **مكوّنان**: Control Node (عليه Ansible، منه التشغيل) + Managed Nodes (تستقبل عبر SSH بلا Agents)، والقناة بينهما Key Pair. بساطة معمارية = سهولة تشغيل.

المعمارية صافية — لكن كيف تتفوق على الطرق التقليدية؟ مقارنة عملية على 3 Servers → **الفصل 3: Comparison**

Comparison — مقارنة طرق إدارة الـ Servers

تخيل أنك مهندس تدير 3 سيرفرات Linux مختلفة التوزيعات — Ubuntu, CentOS, Alpine — وتريد تثبيت Nginx عليها. ثلاث طرق أمامك، والفارق بينها فلسفة كاملة:

وجه المقارنة	Manual 🖱️	Bash Scripts 📄	Ansible ▶️
آلية العمل	الدخول عبر SSH لكل Server وتنفيذ الأوامر يدويًا.	كتابة Script Bash يتعرف على الـ OS وينفذ الأوامر.	كتابة ملف YAML بسيط يُعرّف الـ Desired State للجميع.
السرعة والجهد	بطيئة وتكثر فيها الأخطاء.	تتطلب خبرة عالية في الـ Scripting.	سريعة، مركزية، و Simple Syntax.
Idempotency	غير متوفرة — تدخل يدوي للتأكد.	غير متوفرة تلقائيًا — شروط تُكتب يدويًا.	مدعومة تلقائيًا ✓
دعم التوزيعات	تتطلب معرفة أوامر كل توزيعية (apt , yum , apk)	تتطلب كتابة if/else لكل OS.	الـ Modules تتكفل تلقائيًا ✓

[ASIDE • المفارقة] لاحظ أن Bash Scripts أذكى من Manual لكنها تتراكم: كل توزيعية جديدة = فرع if/else جديد، وكل تحديث = مراجعة كل السكريبتات. Ansible يقلب المعادلة: أنت لا تكتب كيف التنفيذ — أنت تكتب ماذا تريد، وهو يعرف الـ How لكل توزيعية داخل الـ Modules.

◀ TASK [قراءة الجدول عمليًا] reading the table

بمعايير حقيقية: لتثبيت Nginx يدويًا على الثلاثة ستحتاج 3 جلسات SSH و 3 أوامر مختلفة (apt install / yum install / apk add) وتحققًا يدويًا من كل واحد. بـ Bash: Script واحد لكن بشروط تشعبية لكل Package Manager. بـ Ansible: Task واحدة — package: name=nginx state=present — تعمل على الثلاثة، وتحقق الـ Idempotency تلقائيًا في كل Run قادم.

[EXAM 🎯] احفظ أزواج المقارنة: Manual = بطء + أخطاء + لا Bash = Idempotency = مركزي لكن Scripting ثقيل + شروط يدوية . Ansible = مركزي + Idempotency تلقائية + Modules تتكفل بالتوزيعات. هذه المقارنة نفسها سؤال مقابلة متكرر.

🔗 اقتنعنا بالتفوق — أين تُستخدم فعليًا؟ المجالات الثلاثة الكبرى → **الفصل 4: Use Cases**

ثلاثة ميادين رئيسية تشغل فيها Ansible اليوم — الأول هو الأشهر والأهم، والثاني يحمل نقاشًا معروفًا في المجال.

4.1 Configuration Management — إدارة الإعدادات

الاستخدام الأشهر والأهم: تثبيت وتحديث الـ **Packages**، ضبط الـ **Files**، وتحديث الـ Servers من مكان واحد — سواء كانت الـ Servers في Datacenter محلي أو على الـ Cloud. مئات الـ Command، Servers واحدة.

4.2 Infrastructure Provisioning — بناء البنية التحتية

إنشاء وتدمير الموارد السحابية (**EC2, S3, Containers**) عبر الـ **APIs** للخدمات السحابية (AWS, Azure, GCP).

[ASIDE • تنويه من المجال] يُفضّل كثيرون استخدام **Terraform** للـ Provisioning (بناء الـ Infrastructure)، واستخدام **Ansible** للـ Configuration Management (إدارة ما داخل الـ Servers). فصل الاختصاص: الـ Contractor يبنّي، والفني يجهّز من الداخل. 🛠️

4.3 Network Automation — أتمتة الشبكات

ضبط وإدارة الـ **Routers**، الـ **Switches**، والـ **Firewalls** برمجياً عبر بروتوكولات الأتمتة — بدل الدخول لكل جهاز شبكة يدوياً عبر واجهته.

TASK [خريطة القرار السريعة] which tool when ؟

مهمتك	الأداة الأنسب	لماذا
تثبيت Nginx على 20 Server	Ansible	Configuration داخل الأجهزة — مجالها الأول
إنشاء VPC و 10 Instances على AWS	Terraform	Provisioning سحابي عبر API
مشروع كامل: بناء + تهيئة	الاثنان 🟡	Terraform يبنّي → Ansible يهيئ
تحديث Firmware لـ Switches	Ansible	Network Automation — مجالها الثالث

لاحظ: المقارنة ليست «الأفضل مقابل الأسوأ» بل «الأداة في مجالها» — Ansible يستطيع Provisioning، لكن Terraform أقوى فيه.

EXAM 📝 احفظ الثلاثية بالترتيب: **Configuration Management** (الأشهر) · **Infrastructure Provisioning** (سحابي، ونقاش Terraform) · **Network Automation** (Routers/Switches/Firewalls). سؤال «ما استخدامات Ansible؟» — هذه هي الإجابة النموذجية.

🔗 المجالات واضحة — الآن نُجهّز العدة: التثبيت على كل الأنظمة → **الفصل 5: Installation**

Installation — التثبيت وإعداد البيئة

يتم تثبيت Ansible فقط على الـ **Control Node** — قاعدة لا استثناء لها. اختر طريقك حسب نظامك:

Ubuntu / Debian — apt

```
$ sudo apt update && sudo apt install ansible -y
```

macOS — Homebrew

```
$ brew install ansible
```

Windows — WSL

استخدم بيئة **WSL** (Windows Subsystem for Linux) لتشغيل Ubuntu داخل Windows وتثبيت Ansible عليه بشكل طبيعي — ثم طبق أمر Ubuntu أعلاه.

Any OS — Python Pip

```
$ pip3 install ansible
```

TASK [التحقق من التثبيت] verify installation

```
$ ansible --version
ansible [core 2.16.3]
  config file = /etc/ansible/ansible.cfg
  python version = 3.10.12
```

رقم الـ Version ظهر = التثبيت نجح وأنت جاهز.

TASK [إضافات VS Code المقترحة] editor setup

Ansible Extension — Red Hat

YAML Extension — Red Hat

Syntax Highlighting و Validation داخل المحرر — تكتشف أخطاء الـ Indentation قبل أن تصل للـ Run.

[WATCH OUT] خطأ شائع: تثبيت Ansible على الـ Managed Nodes «احتياطيًا». لا فائدة منه إطلاقًا — هي تستقبل الأوامر عبر SSH فقط. التثبيت للـ Control Node وحده.

[DEBUG] `pip3 install` هو الخيار المشترك لكل الأنظمة — مفيد خاصة إذا كان مدير الحزم في توزيعتك يقدم Version قديمًا.

EXAM 🎓 ثلاث طرق تثبيت + **WSL** (Windows) · **Any OS** · `pip3` · **Ubuntu/Debian** · `apt` · **macOS** · `brew`. Windows: وسؤال «أين يُثبت Ansible؟» — **Control Node فقط**.

الأداة جاهزة — لكن الاتصال بالـ Servers يحتاج تأمينًا: الـ SSH Keys → **الفصل 6**

SSH Keys — إعداد الاتصال الآمن بدون Password

لكي تتمكن من الاتصال بال Managed Nodes وتنفيذ ال Instructions تلقائيًا، يُفضّل إعداد الاتصال عبر مفاتيح ال SSH — ال **Public/Private Key Pair**.

[Play - ثلاث Steps واكمل الاتصال]



6.1 إنشاء ال Key Pair على ال Control Node

```
$ ssh-keygen -t rsa -b 4096
```

سيتم إنشاء ملفين في المجلد `~/.ssh/`:

id_rsa.pub — Public Key

ال **Public Key** — هو الذي يتم رفعه إلى ال Managed Nodes. قفله لا يفتح شيئًا.

id_rsa — Private Key

ال **Private Key** — يُحفظ **بسرّية تامة** على ال Control Node ولا يُشارك مع أي أحد أبدًا.

6.2 نسخ ال Public Key إلى ال Managed Node

```
$ ssh-copy-id -i ~/.ssh/id_rsa.pub ubuntu@<MANAGED_NODE_IP>
```

6.3 اختبار الاتصال بدون كلمة سر

```
$ ssh ubuntu@<MANAGED_NODE_IP>
Welcome to Ubuntu 22.04 LTS ... # logged in – no password asked ✓
$ exit
```

[**DEBUG • AWS EC2**] في حالة استخدام أجهزة AWS EC2، يتم استخدام ملف ال Private Key بصيغة `.pem`. المُقدّم من AWS لتنسيق الاتصال الأول — وال User غالبًا `ubuntu` أو `ec2-user` حسب ال OS.

[**WATCH OUT**] خطأ شائع: نسخ `id_rsa` (ال Private) بدل `id_rsa.pub` (ال Public) للسفير = لن يعمل الاتصال. ونسخ ال Private عبر قنوات غير آمنة = ثغرة أمنية كاملة. ال Public يسافر، ال Private لا يتحرك أبدًا.

EXAM 🎓 التسلسل الحرفي: `ssh-copy-id -i ~/.ssh/id_rsa.pub ubuntu@IP` ← `ssh-keygen -t rsa -b 4096` ← اختبار `ssh` `ubuntu@IP`. ومن هذه اللحظة تصبح كل أوامر Ansible القادمة صامتة تمامًا — لا Password بينك وبين الأتمتة.

🔗 القنوات مفتوحة — الآن نسلم Ansible «عدته» وخريطته: **الفصل 7: Modules**

Modules — وحدات التنفيذ

تعتمد جميع مهام Ansible على الـ **Modules** — وهي أدوات برمجية جاهزة تؤدي وظائف محددة. أنت لا تكتب Scripts، أنت تستدعي Modules وتعطيها الـ State المطلوبة.

TASK [أشهر الـ Modules الأساسية] core modules

Module	وظيفته
apt / yum	لإدارة وتثبيت وتحديث وحذف الـ Packages (حسب الـ Distribution).
ping	لاختبار الاتصال والـ Readiness بين الـ Control Node والـ Managed Nodes.
service	لإدارة الـ Services — start, stop, restart.
copy	لنسخ الـ Files من الـ Control Node إلى الـ Managed Nodes.
command / shell	لتنفيذ أوامر النظام المباشرة على الـ Servers.

الـ Modules بالمئات تغطي كل شيء تقريبًا — ويمكنك دائمًا الرجوع للوثائق الرسمية (**Ansible Documentation**) لمعرفة الـ Module المناسب وخياراته لكل Task.

TASK [الـ Modules الثلاثة التي ستستخدمها أولًا] start here

service يتحكم بحياة الـ Service: <code>started</code> , <code>stopped</code> , <code>restarted</code> — <code>enabled</code> عند الـ Boot.	apt / yum قلب إدارة الـ Packages. مع <code>state=present</code> يصبح الأمر <code>Idempotent</code> تلقائيًا.	ping أول Module تجرّبه — يثبت أن الاتصال يعمل. الرد الناجح دائمًا: <code>"pong"</code> .
--	--	--

[KEY IDEA] الـ Module يخفي عنك الـ How: أنت تقول `apt: name=nginx state=present` — والـ Module يعرف بنفسه هل ينقذ `apt-get install` فعليًا أم أن كل شيء موجود فيتجاهل. هذا هو جذر الـ `Idempotency`.

[WATCH OUT] `command` و `shell` قد يكونان فخًا: بهما تستطيع تجاوز كل أنيقة Ansible وكتابة أوامر خام غير `Idempotent`. استخدمهما كملجأ أخير عندما لا يوجد Module مخصص لمهمتك.

EXAM 🚩 احفظ الخمسة الأساسيين بوظائفهم: `copy` (Files) · `service` (Services) · `ping` (Connectivity) · `apt/yum` (Packages) · `command/shell` (Direct commands). سؤال «ما الـ Module الذي تستخدمه لـ X؟» — من هذه القائمة تأتي الإجابات.

العدة جاهزة — نحتاج الآن قائمة الأهداف: من سننقذ عليهم؟ → **الفصل 8: Inventory**

Inventory — ملف ال Managed Nodes

ملف ال **Inventory** هو الملف الذي يحتوي على ال **IPs** الخاصة بال Managed Nodes — مقسمة إلى **Groups** (مجموعات) تستهدفها في الأوامر.

TASK [مثال لملف hosts.ini] example inventory

```
# hosts.ini
[web]
192.168.1.101 ansible_user=ubuntu
192.168.1.102 ansible_user=ubuntu

[db]
192.168.1.201 ansible_user=ubuntu
```

[DEBUG • تشرح الملف] [web] و [db] = أسماء ال Groups بين أقواس مربعة • الأسطر تحتها = ال IPs الأعضاء •
 = ansible_user=ubuntu ال SSH User المستخدم للاتصال بذلك ال Server — تُعرّف مرة هنا بدل تكرارها في كل Command.

TASK [كيف نستهدف؟] targeting

all → 3 Servers 1 Server → [db] 2 Servers → [web]

ansible web ... يضرب Servers الويب فقط • ansible all ... يضرب الجميع • استهداف بالمجموعة بدل حفظ وحشو ال IPs يدويًا كل مرة.

[KEY IDEA] ال Inventory هو «مصدر الحقيقة» لخريطة بنيتك: أضف Server؟ سطر واحد في ال File. غيّرت User؟ تعديل واحد. كل ال Commands وال Playbooks القادمة سٌحيل لهذا الملف عبر -i hosts.ini.

[WATCH OUT] أسماء ال Groups حساسة تمامًا: [web] ليست [webs] ولا [Web]. خطأ حرف واحد في الاستهداف = «لا توجد Hosts مطابقة» — رسالة ال Error لن تنبّهك لل Typo.

EXAM ال Inventory = ملف يحوي ال IPs ال Managed Nodes مقسمة إلى Groups • ال [group_name] Syntax: ثم الأسطر •
 ansible_user يعرّف ال SSH User • يُمرّر دائمًا ب -i.

لدينا ال Modules وال Inventory — وقت أول تنفيذ حقيقي، بلا أي File: → **الفصل 9: Ad-hoc Commands**

التنفيذ الفوري — Ad-hoc Commands ⚡

تُستخدم الـ Ad-hoc Commands لتنفيذ مهمة سريعة بدون الحاجة لكتابة **Playbook** كامل — سطر واحد مباشرة من الـ Terminal. هذا هو الجانب Imperative الذي ذكرناه في الفصل الأول.

TASK [الصيغة العامة] general syntax

```
ansible <group_or_all> -i <inventory_file> -m <module_name> -a "<module_arguments>"
```

TASK [الأمثلة العملية الثلاثة] practical examples

1 فحص الاتصال — ping لكل الـ Servers:

```
$ ansible all -i hosts.ini -m ping
192.168.1.101 | SUCCESS => { "ping": "pong" }
192.168.1.102 | SUCCESS => { "ping": "pong" }
192.168.1.201 | SUCCESS => { "ping": "pong" }
```

2 معرفة الـ Uptime لـ Servers الويب — المجموعة web:

```
$ ansible web -i hosts.ini -a "uptime"
```

[DEBUG] ملاحظة ذكية: عند عدم تحديد `-m` يتم استخدام الـ Module `command` افتراضيًا — لهذا كتبنا `uptime` مباشرة في `-a` دون تحديد Module.

3 التأكد من تثبيت Nginx على جميع الـ Servers:

```
$ ansible all -i hosts.ini -m apt -a "name=nginx state=present" --become
```

[KEY IDEA] `--become` = صلاحيات root (مثل sudo) — الـ Installation يحتاجها دائمًا. و `state=present` هي لغة الـ Idempotency: «تأكد أنه موجود» — موجود أصلاً؟ لا شيء يحدث إطلاقًا.

[WATCH OUT] متى Ad-hoc ومتى Playbook؟ Ad-hoc لمهمة سريعة ولن تتكرر — فحص، Restart، استعلام. أول ما تشعر أن الـ Command سيُعاد لاحقًا أو يُشارك مع الفريق → انقله فورًا إلى Playbook (الفصل القادم).

📌 EXAM احفظ الهيكل بالترتيب: **Become** ← **Arguments (-a)** ← **Module (-m)** ← **Inventory (-i)** ← **Target**. وسؤال الفهم الكلاسيكي: لماذا لم نكتب `-m command` في مثال uptime؟ — لأنه الـ Module الافتراضي.

🔗 الأوامر المنفردة تنجز Task واحدة — السيناريوهات الكاملة تحتاج File منظّمًا. ولغته أولًا: → **الفصل 10: YAML**

تعتمد لغة YAML على الـ **Indentation** (المسافات البادئة) — ليست لغة برمجة بل طريقة مرتبة لوصف الـ Data. إتقان قواعدها = كتابة Playbooks بلا صداع.

[WATCH OUT • القاعدة الأهم] لا تستخدم **Tab** أبدًا — استخدم مسافتين (**Spaces**) دائمًا. خلط Tab مع Spaces هو أكثر سبب لفشل ملفات YAML عند المبتدئين، والـ Error يظهر بعيدًا عن سطره الحقيقي فيصعب تتبعه.

TASK [القاعدة الأولى]

document start

تبدأ الملفات دائمًا بثلاث شرطات `---` — إعلان بداية الـ YAML Document:

```
---
```

TASK [هياكل البيانات الثلاثة]

data structures

③ List

```
skills:
- Linux
- Ansible
- Docker
```

قائمة — شرطة لكل عنصر.

② Dictionary

```
employee:
  name: Mohamed
  age: 30
  is_manager: true
```

قاموس — مفاتيح متعددة تحت مفتاح أب.

① Key-Value

```
name: Ahmed
```

مفتاح وقيمة — أبسط بنية.

[KEY IDEA] الـ Playbook الذي سنكتبه في الفصل القادم ليس شيئًا جديدًا — هو قائمة من القواميس (**List of Dictionaries**) بالضبط: الـ Play عنصر في List، وكل Task داخله Dictionary. من فهم هذه الهياكل الثلاثة، فهم Ansible كله.

[WATCH OUT] في الـ Dictionary: لاحظ أن `name` و `age` بادئة بمسافتين تحت `employee` — هذا الـ Indentation هو ما يجعلهما داخله. لو صنفتهما بمستواه لصارا Key-Value مستقلين منفصلين تمامًا.

EXAM قواعد YAML الثلاثة الذهبية: ① ابدأ بـ **Spaces** ② `---` لا **Tabs** ③ ثلاث هياكل فقط: **Key-Value, Dictionaries, Lists**. كل تعقيد لاحق في الـ Playbooks هو تركيب لهذه الثلاثة.

📖 اللغة بين يديك — الوقت جاء: أول Playbook حقيقي كامل → **الفصل 11**

ال **Playbook** هو ملف YAML يحتوي على مجموعة من ال **Plays**، وكل Play تحتوي مجموعة من ال **Tasks** — لتنفيذ سيناريو Automation كامل على مجموعة Servers محددة. هذا هو قلب Ansible النابض.

[Hierarchy - التسلسل الهرمي]

Modules

apt · service

◀ تستدعي

Tasks

steps

◀ تحتوي

Plays

hosts · become

◀ يحتوي

Playbook

site.yml

◀ TASK [مثال: Playbook لتثبيت وتشغيل Apache]

أنشئ ملف site.yml

```
---
- name: Install and Start Apache Web Server
  hosts: web
  become: yes

  tasks:
    - name: Install Apache Package
      apt:
        name: apache2
        state: present

    - name: Ensure Apache Service is Running and Enabled
      service:
        name: apache2
        state: started
        enabled: yes
```

◀ TASK [تشغيل ال Playbook] execute it

```
$ ansible-playbook -i hosts.ini site.yml
PLAY [Install and Start Apache Web Server]
TASK [Gathering Facts] ok: [192.168.1.101]
TASK [Install Apache Package] changed: [192.168.1.101]
TASK [Ensure Apache Service is Running and Enabled] changed: [192.168.1.101]
PLAY RECAP
192.168.1.101 : ok=3 changed=2 failed=0
```

[DEBUG • قراءة سطرًا بسطر] → استهدف ال Group web من ال become: yes · Inventory → نفذ بصلاحيات root · كل Task = Name + وصف + State + Module مطلوبة. لاحظ الصياغة: لا نقول «ثبتت» بل «اجعله present».

[PROOF] شغله مرة ثانية الآن. النتيجة: changed=0 وكل ال Tasks ok — ال Idempotency حيّة أمام عينيك: ال State متحققة فلا تغيير.



🔍 ثبتنا وشغلنا — الإزالة النظيفة وقراءة النتائج باحتراف: → **الفصل 11-b**

نفس القوة التي تُثبت تُزيل — بعكس الـ State فقط. ثم نتقن قراءة مؤشرات النتائج الثلاثة.

TASK [Apache لإيقاف وحذف] remove playbook

```
---
- name: Stop and Remove Apache Web Server
  hosts: web
  become: yes

  tasks:
    - name: Stop Apache Service
      service:
        name: apache2
        state: stopped

    - name: Remove Apache Package
      apt:
        name: apache2
        state: absent
```

[KEY IDEA] الترتيب العكسي المنطقي: **نوقف الـ Service أولاً** ثم نحذف الـ Package — حذف حزمة والـ Service الخاصة بها تعمل = سلوك غير متوقع. وكلمة السر كلها في الـ `state : present / absent / started / stopped` — أربع Values تحكم العالم.

TASK [مؤشرات النتائج] result indicators

المؤشر	اللون	المعنى
ok	أخضر	الـ Server في الـ Desired State بالفعل — لم يتطلب تغييرًا.
changed	أصفر	تم إجراء الـ Change بنجاح على الـ Server.
failed	أحمر	حدث Error في الـ Task — توقف التنفيذ على ذلك الـ Server.

[RESCUED • لو ظهر failed] اقرأ اسم الـ Task الأحمر أولاً — يخبرك أي **Step** فشلت، ثم السطر أسفلها — يخبرك لماذا (شبكة؟ Permissions؟ اسم Package خاطئ؟). 90% من الأعطال تُحل من هذه السطور دون أي بحث خارجي.

EXAM الفرق الجوهرية بين ok وchanged هو **جوهر فلسفة Ansible: ok** = الـ State متحققة مسبقاً (الـ Idempotency تعمل)، changed = عمل حقيقي تم الآن. وقراءة failed تبدأ من اسم الـ Task + الـ Error Message تحته.

File واحد أنجز سيناريو كاملاً — لكن مع كبر الـ Projects يصير وحشًا. التنظيم النهائي: → **الفصل 12: Roles & Galaxy**

عندما تكبر مشاريع الـ Automation، يصبح وضع جميع المهام والتعليمات في ملف Playbook واحد **صعب الصيانة**. تعطينا **Ansible Roles** هيكلية مجلدات قياسية لتنظيم الـ Project وتجزئته إلى مكونات **Modular & Reusable**.

TASK [الهيكل التنظيمي للـ Role] role structure

```
my_role/
├─ tasks/      # الرئيسية Tasks الـ - main.yml
├─ handlers/   # Tasks المشروطة الـ (Handlers)
├─ templates/  # Jinja2 قوالب الـ
├─ files/      # الملفات الثابتة المراد نقلها
├─ vars/       # Role الخاصة بالـ Variables
├─ defaults/  # الافتراضية Variables الـ
└─ meta/      # Role معلومات عن الـ
```

[KEY IDEA] الـ **Encapsulation** = Role — تغليف. بدل Playbook من 200 سطر، تستدعي Role باسمه، وكل من يقرأ الـ Project يفهم الـ Structure فورًا لأن الجميع يعرف هذا الهيكل القياسي.

TASK [منصة Ansible Galaxy] galaxy

مستودع رسمي مفتوح المصدر يُمكنك من **مشاركة وتنزيل الـ Roles** والـ **Collections** الجاهزة المبنية من قبل مجتمع Ansible — لتسريع عمليات الـ Development والأتمتة. بدل بناء Role لـ Apache من الصفر: ابحث في Galaxy، نزل، وعدّل لاحتياجك.

Community-driven

Collections

Ready-made Roles

galaxy.ansible.com

[WATCH OUT] متى تكتب Role بدل Playbook؟ قاعدة عملية: أول Playbook لك = Playbook عادي. عندما يبدأ الملف بالتكرار أو تجد نفس الـ Tasks في Projects متعددة = وقت التجزئة إلى Roles. لا تبدأ بالتعقيد.

EXAM اجمع رسالة الكتاب كلها في جملة — سؤال الـ Interview الختامي: «ما Ansible؟» → أداة **Agentless Automation** عبر SSH، بـ **YAML Declarative Syntax** للـ Desired State، تنسم بالـ **Idempotency**، تنظم عملها بـ Inventory و Playbooks وتتوسع بـ Roles و Galaxy.

❖ ختامًا

هذا الدليل يلخص جميع الأساسيات والمفاهيم العملية التي تم شرحها في الدورة التعليمية لقناة **Cloud Code** — ويمكن الاعتماد عليه كمرجع سريع ومكتوب أثناء كتابة وتنفيذ سيناريوهات الأتمتة. ثم طبق: Virtual Machine كـ Server + Control Node، وأعد بناء كل الأمثلة بيدك — الـ Automation تُتقن بالممارسة. ▶

اختبر نفسك — 12 سؤالاً

غَطِّ الإجابات حتى تحل الاثني عشر كلها. +10 صحيح = جاهز عمليًا.

7. أي Key يُرفع إلى الـ Managed Node — وماذا يحدث لو رفعت الآخر؟

8. اذكر وظائف الـ Modules الخمسة الأساسية: apt/yum, ping, .service, copy, command/shell

9. ماذا تعبر عنه المجموعات في الـ Inventory مثل [web]؟

10. ما الـ Module الافتراضي في Ad-hoc عند حذف -m؟

11. ماذا تعني changed=0 في PLAY RECAP بعد التشغيل الثاني؟

سؤال الشرف: مميزات Ansible الأربع + مجالاته الثلاثة + جملة التعريف الكاملة — من الذاكرة.

1. ما المجالان الأساسيان اللذان تُستخدم فيهما Ansible؟

2. ما المتطلب الوحيد على الـ Managed Nodes في Linux؟

3. على أي Node يُنَبِّت Ansible؟

4. عرّف Idempotency بجملة واحدة مع مثال.

5. ما الفرق بين Declarative و Imperative في Ansible؟

6. اكتب Command لإنشاء SSH Key Pair بطول 4096.

[DEBUG] الأسئلة تغطي الفصول الـ 12 بالتسلسل. أي تعثر → عد لفصلها من **الفهرس** قبل فتح الإجابات.

مفتاح الإجابات + التقييم ✓

#	الإجابة	السبب
1	Configuration Management + Infrastructure Provisioning	المجالان الأساسيان — الثالث: Network Automation
2	Python فقط	مثبتة افتراضيًا في أغلب التوزيعات — لا Agents
3	Control Node فقط	ال Managed Nodes تستقبل عبر SSH دون تثبيت
4	تنفيذ نفس ال Script مرارًا لا يغيّر شيئًا إلا عند الحاجة الفعلية — مثال: Apache مثبت مسبقًا → لا تغيير	ال Desired State متحقق → ok
5	Declarative في ال Playbooks (تحدد ال State) · Imperative في ال Ad-hoc Commands (أمر فوري)	أسلوبان لموقفين مختلفين
6	<code>ssh-keygen -t rsa -b 4096</code>	t- النوع (rsa) · b- الطول (4096 bit)
7	id_rsa.pub — Public Key · رفعت ال Private؟ لن يعمل الاتصال + ثغرة أمنية	ال Public يُرفع، ال Private يبقى سرّيًا
8	apt/yum = Packages · ping = Connectivity · service = Services (start/stop/restart) · copy = Files · command/shell = Direct commands	الخمسة الأساسية — كل شيء آخر يُبنى عليها
9	ال IPs الخاصة بال Managed Nodes — ال Groups = وحدات Targeting بنفس اسم المجموعة	نستهدف [web] أو all بدل ال IPs
10	command	الافتراضي عند غياب m-
11	ال State كانت متحققة أصلاً — Idempotency عملت 🔄	نفس ال Playbook مرتان = صفر Changes

[**PROOF** · 🔥 سؤال الشرف] المميزات الأربع: **Agentless · Simple YAML Syntax · Declarative Approach · Idempotency** —
والمجالات الثلاثة: **Configuration Management · Infrastructure Provisioning · Network Automation**. والجملة الكاملة: أداة
Agentless Automation عبر SSH، ال Declarative YAML لل Desired State، منظمة ال Inventory و Playbooks، متوسعة ال
Roles و Galaxy. أجبتها من الذاكرة؟ أنت تفهم Ansible لا تحفظه.

TASK [احسب نتيجتك] scoring

> 7 · إعادة

نقذ أمثلة الكتاب يدويًا على lab حقيقي —
الفعل يثبت أكثر من القراءة.

7-9 · متين 🍌

أعد الفصول التي أخطأت فيها ثم أعد
الاختبار.

10-12 · جاهز عمليًا 🏆

انتقل لبناء Roles حقيقية ومشروع
Automation متكامل.

الورقة المرجعية — اطبعها وعلّقها

- ▶ **FULL FLOW:** keygen → ssh-copy-id → inventory → ad-hoc/playbook → recap
- ▶ **ARCHITECTURE:** Control Node —SSH→ Managed Nodes (Agentless • Python only)
- ▶ **STRUCTURE:** Playbook → Plays → Tasks → Modules • scale up: Roles → Galaxy

[Playbook Structure 📄]

[Setup Commands ⚙]

```
---
- name: scenario name
  hosts: web
  become: yes
  tasks:
    - name: task name
      apt:
        name: apache2
        state: present

# run:
ansible-playbook -i hosts.ini site.yml
```

```
$ brew install ansible
$ sudo apt update && sudo apt install ansible -y
$ pip3 install ansible
$ ansible --version
$ ssh-keygen -t rsa -b 4096
$ ssh-copy-id -i ~/.ssh/id_rsa.pub user@ip
$ ssh user@ip
```

[Ad-hoc Syntax ⚡]

```
ansible <group> -i hosts.ini -m <module> -a "args"
ansible all -i hosts.ini -m ping
ansible web -i hosts.ini -a "uptime"
ansible all -m apt -a "name=nginx state=present" --become
```

[State Values 🎆]

Package: موجود / محذوف	present / absent
Service: تعمل / متوقفة	started / stopped
تشتغل عند ال Boot	enabled: yes
متحقق ok • تغيير changed • خطأ failed	

my_role/ tasks/ • handlers/ • templates/ • files/ • vars/ • defaults/ • meta/ # + galaxy.ansible.com

Roles + Galaxy

Inventory Groups

Idempotent

Declarative

Simple YAML

Agentless

KEY IDEA • الجملة الختامية [أنت تكتب ال **Desired State** مرة واحدة — و **Ansible** يحققها على كل ال **Managed Nodes**، ولا يلمس شيئاً إن لم يكن التغيير ضرورياً. هذه هي ال Automation كما يجب أن تكون. ▶]